COMP90038 Algorithms and Complexity

Transform-and-Conquer

Junhao Gan

Week 7 Lecture 14

Semester 1, 2026

Transform and Conquer

- Instance simplification
- Representational change
- Problem reduction

Instance Simplification

General Principle: Try to make the problem easier through some pre-processing, e.g., sorting.

As we will see next, we can pre-sort input to speed up, for example

- uniqueness checking
- finding the most frequent elements

Uniqueness Checking, Brute-Force

Uniqueness Checking Problem:

Given an unsorted array $A[0] \ldots A[n-1]$, decide whether all $A[i]$'s are distinct.

The obvious approach is brute-force:

```
for i ← 0 to n − 2 do
    for j ← i + 1 to n − 1 do
        if A[i] = A[j] then
            return False
return True
```

What is the complexity of this?

Uniqueness Checking, with Presorting

Sorting makes the problem easier:

```
SORT(A, n)
for i ← 0 to n − 2 do
    if A[i] = A[i + 1] then
        return False
return True
```

What is the complexity of this?

Exercise: Computing a Mode

A mode is a *most frequent* element in a given array.

For example, in

$$42, 78, 13, 13, 57, 42, 57, 78, 13, 98, 42, 33$$

both elements 13 and 42 are modes.

Finding a mode: Given an array A of n items, find a mode.

Discussion: Any brute-force approach can you think of?

What about a pre-sorting approach?

Mode Finding, with Presorting

```
SORT(A, n)
i ← 0
maxfreq ← 0
while i < n do
    runlength ← 1
    while i + runlength < n and A[i + runlength] = A[i] do
        runlength ← runlength + 1
    if runlength > maxfreq then
        maxfreq ← runlength
        mode ← A[i]
    i ← i + runlength
return mode
```

Again, after sorting, the rest takes linear time.

Searching, with Presorting

Dictionary Search Problem:

Given an unsorted array A, find item x (or determine that it is absent).

Compare these two approaches:

- Perform a sequential search
- Sort, then perform binary search

What are the complexities of these approaches?

Searching, with Presorting

What if we need to search for m items?

Let us do some calculation (consider the worst case for simplicity):

Take $n = 1024$ and $m = 32$.

Sequential Search:

$$m \times n = 32 \times 2^{10} = 32,768\,.$$

Sorting + Binary Search:

$$n \log_2 n + m \times \log_2 n = 10 \times 2^{10} + 32 \times 10 = 10,560\,.$$

Average-case analysis will look somewhat better for sequential search, but pre-sorting will still win.

Exercise: Finding Anagrams

An anagram of a word w is a word which uses the same (multi-)set of letters of w but in a different order.

Example: 'ate', 'tea' and 'eat' are anagrams.

Example: 'post', 'spot', 'pots' and 'tops' are anagrams.

Example: 'garner' and 'ranger' are anagrams.

Exercise: Finding Anagrams

You are given a list of n words $\{w_1, w_2, \ldots, w_n\}$, each of which consists of $O(1)$ letters.

Design an algorithm to group all the words that are anagrams to each other.

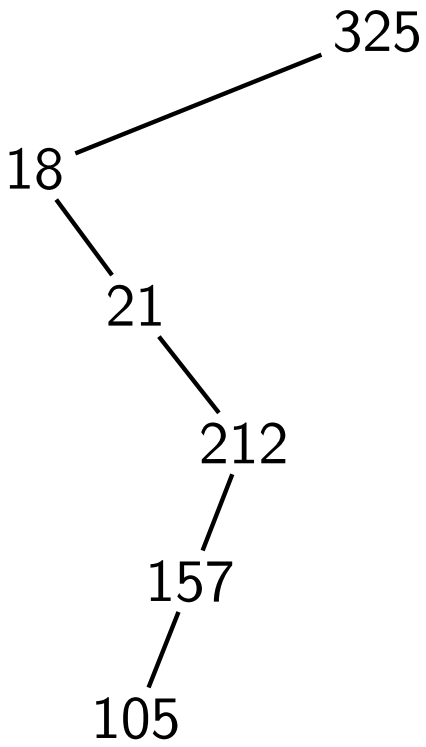
Binary Search Trees

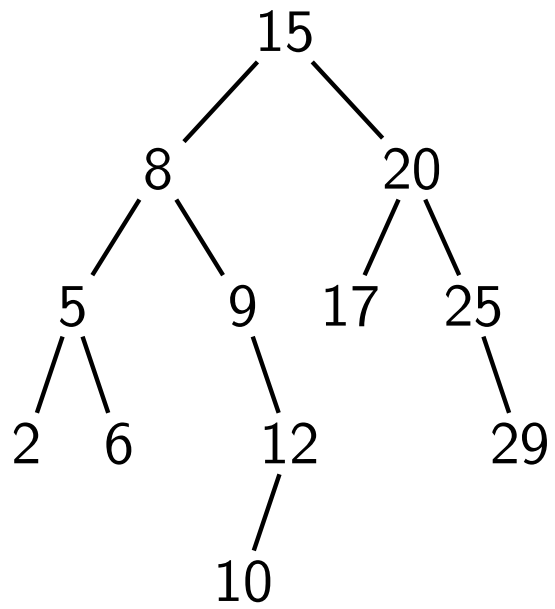
A **binary search tree** (BST) on a set S of n key values is a binary tree satisfying the followings:

- Each key value occurs in at least one node of the tree.
- Each node distinguishes its children: the left child and the right child;
- For any node v in the tree, the key of v is:
  - larger than *any* key of the nodes in its left sub-tree and,
  - smaller than *any* key of the nodes in its right sub-tree.

Binary Search Trees

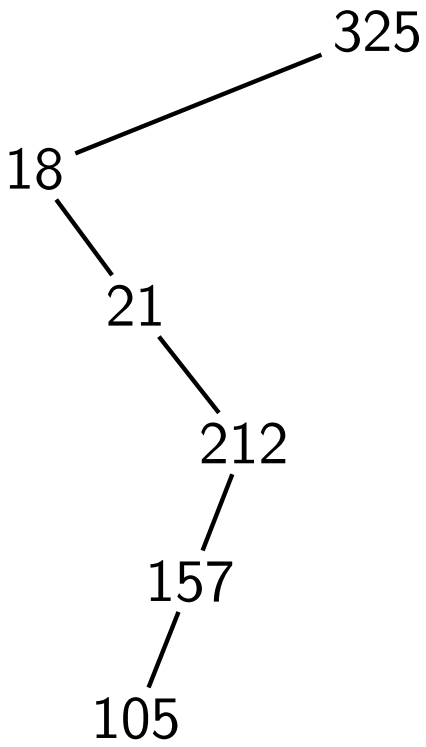
Question: Which of the followings are BST's?

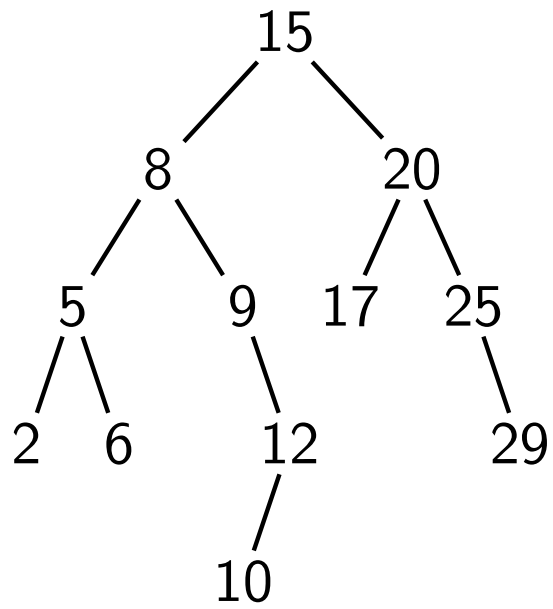




Binary Search Trees

Question: How do you search a given key k in a BST?





Searching on a Binary Search Tree

Search(k):

- if the current tree is empty, return **NULL**;
- if $k = r$, where r is the key of the root, then return the root and done;
- if $k < r$, recursively search k in the left sub-tree;
- if $k > r$, recursively search k in the right sub-tree;

Searching on a Binary Search Tree

The worst-case searching time of a BST actually **depends on** its height.

To reduce the worst-case searching time, we need to reduce the height of the tree, equivalently, to **"balancing"** the tree.

When the BST also needs to support key insertions and deletions, this is particularly crucial.

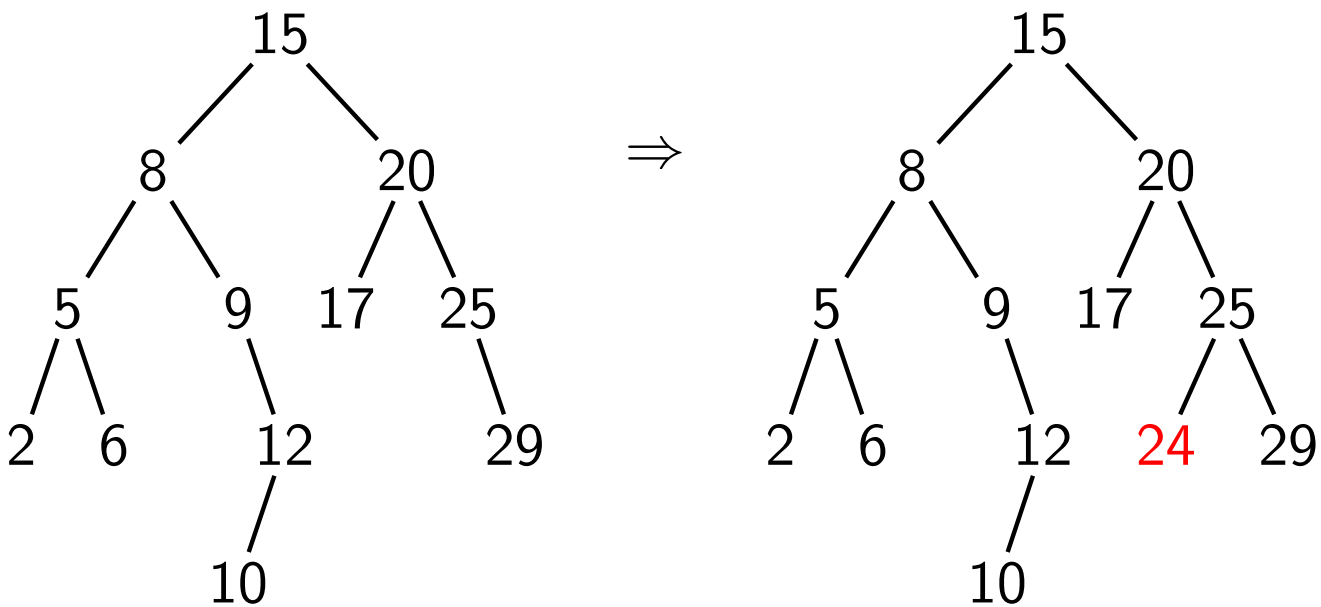
With different **strategies** to maintain the balance, different balanced BSTs are defined, such as: **AVL Tree**, **Red-black Tree**, **(a, b)-Tree**, and etc.

Insertion in a Binary Search Tree

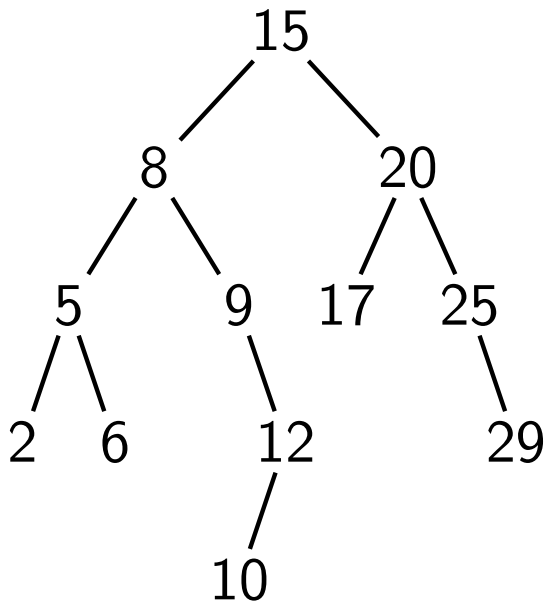
To insert a new element k into a BST, we pretend to search for k.

When the search has taken us to the fringe of the BST (we find an empty sub-tree), we insert k where we would expect to find it.

Inserting 24:



BST Traversal Quiz



Performing ..... traversal of a BST will produce its elements in sorted order.

Next Up: Balancing Binary Search Trees

As aforementioned, to optimise the performance of BST search, it is important to keep trees (reasonably) balanced.

Next lecture, we shall look at AVL trees and 2–3 trees.